

EquityFlow

Interview Preparation — Technical Deep Dive

May 14, 2026

A code-grounded interview preparation document for the local Indian-market paper-trading workstation.

Generated from the current repository contents

Table of Contents

1. Project Overview	3
2. Full Tech Stack Breakdown	4
3. System Architecture	6
4. Key Technical Decisions	8
5. Deep Dive - Hardest Problems	9
6. Interview Questions and Answers	11
7. Follow-up Questions	14
8. Quick Reference Card	16

1. Project Overview

EquityFlow is a local paper-trading workstation for Indian markets. It combines a **Next.js** terminal-style trading desk with a **FastAPI** market-data proxy so a user can watch NSE equities, **F&O** underlyings, MCX-style commodities, indices, charts, option chains, alerts, strategies, and a simulated portfolio from one interface.

The problem it solves is practical: a student or trader can practice market workflows without risking money and without scattering the workflow across a charting site, a broker order book, spreadsheets, and custom scripts. The frontend owns the paper account and trading simulation, while the backend hides provider details, normalizes quotes, coalesces network requests, and streams live or fallback prices to the UI.

Real-world domain and use case

- Domain: **Indian equity and derivatives market simulation**, including NSE cash equities, NSE F&O contracts, index data, and commodity-style instruments.
- Primary user: someone preparing or practicing trades who wants a dense terminal, chart matrix, order ticket, portfolio ledger, and strategy/risk view without executing live broker orders from the browser engine.
- Core workflow: pick or search a symbol, inspect live quote/candle/depth data, place a simulated **MARKET**, **LIMIT**, **SL**, or **SL-M** order, then track open orders, positions, risk, realized P&L, and strategy attribution.
- Operational workflow: connect Upstox OAuth or Groww credentials in the backend environment, start FastAPI on port 8001, start Next.js on port 3000, and use diagnostics/status panels to see provider health.

Key outcomes and metrics visible in the code

- Default paper account balance is **INR 100,000** from **API_CONFIG.defaultBalance**.
- The instrument master **instruments.csv** has **205,894** rows and is used to resolve F&O trading symbols when simplified tickers are entered.
- Backend provider caches use short live-market TTLs such as **3 seconds** for LTP, **5 seconds** for quotes, **15 seconds** for option chains/greeks, and longer TTLs for candles/fundamentals.
- Backend concurrency is bounded with environment knobs like **GROWW_MAX_CONCURRENT** and **UPSTOX_MAX_CONCURRENT**, defaulting to 4 each.
- The SSE workstation stream sleeps about **3 seconds** while markets are open and **15 seconds** when closed; the broader price stream uses a faster loop around **0.6 seconds** during open sessions.
- The chart matrix is capped at **4 charts**; chart layouts, watchlists, alerts, theme, layout mode, and the paper portfolio are stored in browser **localStorage**.
- Verification assets include **Vitest** tests for engine/risk/parser/replay/cache behavior and a Python unittest for Upstox token-expiry handling.

What is intentionally not present

There is **no SQL database**, no **Prisma** schema, no **Dockerfile**, no **docker-compose** file, and no migration directory in this checkout. The persistent application state is browser **localStorage**, provider token JSON on the backend, static/mock constants, and the CSV instrument master. There is also no ML/AI training or inference pipeline; the replay and option-chain fallbacks are deterministic/math-based simulations, not learned models.

2. Full Tech Stack Breakdown

The stack is split cleanly between a client-heavy trading workstation and an async Python data proxy. The table below separates libraries used in source from dependencies that are declared but not currently referenced by the application code.

Layer	Technology actually present	Where it is used	Why this choice fits the codebase
Frontend framework	Next.js 16.2.6 , React 18 , App Router , TypeScript TSX	src/app routes, layout.tsx , client components, page-level workspaces	Next.js gives file-based routing, image/font handling, production build tooling, and a natural app shell. React client components are necessary because trading, chart, and portfolio state changes continuously in the browser.
Frontend language/tooling	TypeScript 5 , strict tsconfig , path alias @/* , ESLint 9	All src app, component, hook, lib, service, and test files	Strict typing makes order, quote, position, option-chain, and API contracts explicit. It reduces mistakes in a finance UI where field names and numeric semantics matter.
Frontend data fetching	@tanstack/react-query , custom request-cache.ts , EventSource	Providers , useStockData.ts , usePriceStream.tsx , request-cache tests	React Query handles ordinary HTTP queries. The custom cache adds path-aware TTLs, in-flight dedupe, stale-on-429 behavior, and request coalescing. EventSource handles one-way streaming efficiently.
UI system	Tailwind CSS , custom terminal CSS variables, lucide-react , Radix Dialog , Radix Toast , Next Image	globals.css , layout, command palette, provider status, order pad, toasts, cards, tables	The app is a dense terminal/workstation rather than a marketing page, so Tailwind plus CSS variables gives fast theme iteration and compact layouts. Radix is used for accessible modal/toast primitives.
Charts and visuals	lightweight-charts , HTML canvas sparkline, image logo fallbacks	StockChart , Sparkline , StockLogo	Lightweight Charts is a specialized financial chart library with candlesticks, volume, price scales, time scale control, and imperative updates. Canvas sparklines are lightweight for many small cards.
Client trading engine	localStorage paper database, engine.ts , risk-engine.ts , trading-charges.ts	Portfolio state, orders, positions, transactions, analytics, risk, charges, pending-order processing	A local paper engine avoids backend account/auth complexity while still modeling realistic order states, margin, charges, partial fills, stop/limit triggers, and P&L attribution.
Backend framework	FastAPI , Pydantic , Uvicorn , python-dotenv	backend/main.py , backend/requirements.txt , startup.sh	FastAPI matches async I/O, typed request/response models, easy OpenAPI generation, and simple deployment as an ASGI app. Pydantic documents and validates API shapes.
Backend networking	httpx.AsyncClient , asyncio , semaphores, locks, background tasks, StreamingResponse	Provider calls, request caches, in-flight task maps, SSE streaming endpoints	Provider APIs are network-bound and rate-limited. Async HTTP clients with connection pooling avoid per-call TLS setup, while semaphores and coalescing control request pressure.
Market data providers	Upstox OAuth/token flow and API, Groww auth/API fallback, public Groww fundamentals scraping	Backend quote, LTP, OHLC, candles, option-chain, greeks, diagnostics, status routes	Upstox is the preferred normalized market-data provider when available; Groww broadens coverage and provides a fallback path. The app exposes provider status so degradation is visible.
Data assets	instruments.csv , static mock stock/index/commodity lists, token JSON file, browser storage keys	F&O resolution, mock fallback views, provider auth persistence, local account data	A CSV master is practical for symbol lookup without adding a database. Mock data keeps the UI useful when providers are disconnected, rate-limited, or closed.

Layer	Technology actually present	Where it is used	Why this choice fits the codebase
Testing and quality	Vitest , Python unittest , npm run lint , npx tsc --noEmit	Engine/risk/parser/replay/cache tests and Upstox token expiry tests	The highest-risk business logic is unit-testable outside the UI. Static typing and linting catch integration and React mistakes before runtime.
Deployment/de vops	GitHub Actions Azure workflow, startup.sh , start-all.ps1	.github/workflows/main_equityflow.yml , root startup scripts	The repo includes an Azure App Service deployment path for the Python backend and local Windows helper scripts for starting backend/frontend together.
Declared but not referenced in current source	framer-motion , class-variance-authority , several Radix packages beyond Dialog/Toast, @fontsource/inter	Present in package.json ; searches did not find imports in current src	These may be leftovers from earlier UI experiments or future-ready dependencies. In an interview, say they are declared dependencies, not current architectural pillars.

3. System Architecture

High-level architecture

EquityFlow is a two-process local system: the **Next.js** frontend renders the trading workstation and owns paper-account state, while the **FastAPI** backend acts as the market-data, provider-auth, normalization, caching, and streaming layer. The app is intentionally not a traditional three-tier database app. It is closer to a terminal front end plus an async market-data gateway.

Data flow: user interaction in a React page or command palette produces either an HTTP query, an SSE subscription, or a local paper-trading mutation. HTTP queries go through **src/services/request-cache.ts** and **src/services/api.ts** to FastAPI. Streaming quotes flow through one shared **EventSource** to **/api/stream/workstation**. Local paper orders go to **engine.ts**, which persists to **localStorage**, emits a portfolio-store change, and drives portfolio, positions, orders, analytics, and risk components.

Component map

Component	Responsibility	Communication
TerminalWorkspace	Home desk with dense/classic/tabs modes, index ribbon, quote boards, chart matrix, replay tab, alert panel, action rail, portfolio rail, and order-pad launch events.	Consumes streaming prices and local portfolio store; dispatches custom events for chart/order workflows.
OrderPad	Paper ticket for BUY/SELL, DELIVERY/INTRADAY, MARKET/LIMIT/SL/SL-M, quantity or lot selection, strategy tag, estimated charges, and required margin.	Calls usePortfolio.placeOrder and displays Radix dialog/toast feedback.
engine.ts	Local paper-trading engine: validation, margin reservation, stop/limit trigger checks, partial fills, position averaging, sell reduction, realized/unrealized P&L, analytics, and persistence.	Reads/writes browser localStorage key equityflow_db ; exposed by hooks via useSyncExternalStore .
usePriceStream.tsx	Single shared EventSource provider that tracks active subscribers, reconnects with backoff, and publishes ticker, commodity, index, and provider-status maps.	Connects to FastAPI SSE, exposes hooks such as useStreamPrice and useAllStreamPrices .
useStockData.ts	React Query hooks for quote, F&O quote, commodity quote, indices, candles, depth, trending, search, option chain, orders, holdings, positions, status, diagnostics, and stock detail.	Uses API wrapper functions and adapts refetch intervals to market status.
StockChart	Imperative candlestick/volume/indicator chart with interval aggregation, candle caching, IST formatters, ResizeObserver cleanup, live-LTP patching, and disposal-race guards.	Fetches candles over HTTP; updates chart series imperatively with lightweight-charts.
FastAPI backend	Provider auth/status, market data normalization, request dedupe, provider fallback, F&O symbol resolution, order proxy routes, diagnostics, and SSE streams.	Receives frontend HTTP/SSE requests and calls Upstox/Groww with async httpx .
instruments.csv	Instrument master used to resolve simplified F&O tickers to real trading symbols, expiry, lot size, strike, option type, and underlying.	Loaded/indexed by backend helpers, not by the browser.

Backend API design

API group	Endpoints	Shape and purpose
Health and diagnostics	GET /, /api/status, /api/diagnostics	Return API identity, provider order, auth configuration, token source/expiry, cooldowns, cache sizes, last provider errors, and SSE cache status.

API group	Endpoints	Shape and purpose
Upstox auth	GET /api/upstox/auth/url, POST /api/upstox/auth/token, DELETE /api/upstox/auth/token	Generate OAuth URL, exchange authorization code for token, persist runtime token JSON, and clear/disconnect the token.
Market data	/api/quote, /api/ltp, /api/ohlc, /api/candles/{ticker}, /api/depth/{ticker}, /api/sparkline/{ticker}	Normalize live or cached provider data into frontend-friendly quote/candle/depth/sparkline structures.
Derivatives and commodities	/api/option-chain, /api/greeks, /api/fno/resolve, /api/fno/quote/{ticker}, /api/commodity/quote/{ticker}	Resolve F&O symbols, fetch option chains/greeks when possible, and return synthetic fallback option data if live chain data is unavailable.
Orders and broker account	POST /api/order/create, /modify, /cancel, GET /api/order/status/{id}, /list, /detail/{id}, /trades/{id}, /holdings, /positions	Proxy Groww order/account APIs. This is separate from the browser-local paper engine, which simulates trades locally.
Discovery and workstation	/api/search, /api/stocks, /api/indices, /api/trending, /api/stock-details/{ticker}, /api/workstation/snapshot	Feed stock discovery, index ribbons, stock details, and batch snapshot views to reduce frontend request fan-out.
Streaming	/api/stream/workstation, /api/stream/stock/{ticker}, /api/stream/demand, /api/stream/prices	Server-Sent Events return newline-delimited JSON frames using StreamingResponse , no-cache headers, and closed/open-market sleep intervals.

Persistence and schema

- **Browser localStorage schema:** **User**, **Order**, **Position**, **Transaction**, strategy analytics, alerts, watchlists, layout mode, chart layout, and theme. The canonical TypeScript contracts live in **src/lib/types.ts**.
- **Backend Pydantic models:** **StockQuote**, **SearchResult**, **MarketIndex**, **CandleData**, **MarketDepth**, **OptionGreeks**, **UpstoxTokenRequest**, **PlaceOrderRequest**, **ModifyOrderRequest**, **CancelOrderRequest**, and **StockListItem**.
- **Instrument master:** **instruments.csv** carries exchange, exchange token, trading symbol, Groww symbol, instrument type, segment, ISIN, underlying, expiry date, strike price, lot size, tick size, freeze quantity, and buy/sell flags.
- **No relational schema:** because there is no SQL database or migration layer, interviews should frame persistence as a deliberate local-first paper-trading design, not an unfinished ORM setup.

Caching, async, streaming, and event-driven behavior

- The backend uses persistent **httpx.AsyncClient** instances with connection limits so provider calls reuse connections instead of recreating TLS sessions.
- Identical provider GETs are coalesced through in-flight task maps, protected by **asyncio.Lock**, and bounded by **asyncio.Semaphore** for provider-specific concurrency.
- 429 responses set cooldowns, parse **Retry-After**, and prefer stale cached data when available instead of failing the whole UI immediately.
- The frontend request cache mirrors the same idea: path-aware TTLs, in-flight dedupe, stale-on-429, and cache pruning to reduce burst load from many cards/pages.
- The UI uses browser events for cross-component commands such as opening an order ticket or adding a chart from the command palette.
- The portfolio hook uses **useSyncExternalStore** so React views subscribe to a stable external portfolio store rather than copying localStorage state independently.

4. Key Technical Decisions

Decision	Reasoning visible in the code	Trade-off and alternative
Use a FastAPI proxy instead of calling providers from the browser	Provider credentials, OAuth token refresh, token-file persistence, cooldowns, request coalescing, and response normalization all live in backend/main.py .	This keeps secrets out of the frontend and centralizes provider quirks. The trade-off is running a second process. An alternative is Next.js API routes, but Python async plus FastAPI is already the chosen backend shape.
Use Server-Sent Events instead of WebSockets for live prices	The frontend only needs server-to-browser price/status updates. EventSource is simpler than bidirectional WebSockets and reconnects naturally.	SSE is less flexible for client-to-server push. WebSockets would make sense for collaborative trading rooms or server-side order entry, but it is unnecessary for one-way quote streaming.
Keep paper trading state in localStorage	The goal is a local paper desk. The engine persists the portfolio on the client, making it fast, offline-tolerant, and free from backend account setup.	The trade-off is no cross-device sync, server auditing, or multi-user protection. A production version would move orders/account state to Postgres or another durable backend store.
Model real order varieties in the local engine	engine.ts implements MARKET, LIMIT, SL, SL-M , pending orders, partial fills, margin reservation, stop/limit triggers, and status notes.	This is more complex than immediately filling every order, but it makes the interview story stronger because correctness is treated as domain logic, not UI decoration.
Prefer Upstox with Groww fallback	The backend constructs provider order from environment, has Upstox token machinery, Groww auth/checksum/token fallback, and per-provider cooldown/error state.	Multiple providers increase code paths and normalization work, but reduce downtime when one provider is unavailable, rate-limited, or missing a data type.
Use an instrument CSV and derived F&O index	The backend builds simplified derivative lookup keys and sorts contracts by expiry so UI-friendly tickers can resolve to real trading symbols.	CSV lookup avoids a database, but startup/indexing cost and file freshness become concerns. A production alternative is a scheduled instrument-master ingestion job with indexed DB tables.
Expose diagnostics directly in the app	The diagnostics page and provider status panel surface cooldowns, auth fields, last success, last errors, cache sizes, and stream health.	This adds UI code, but it is valuable in a provider-dependent app because silent data failure is worse than visible degradation.
Use a dense terminal design system	Tailwind/CSS variables define terminal panels, rows, badges, grids, table heads, monospaced numbers, and amber/dark themes. The main workspace prioritizes scanning and repeated action.	The UI is less beginner-friendly than a card-heavy dashboard, but it matches the trader workstation domain and handles high information density.
Use lightweight-charts imperatively	The StockChart component manages chart API refs, series refs, ResizeObserver, candle caches, interval aggregation, and disposal-race handling.	Imperative chart APIs need careful cleanup, but a specialized finance chart library is more correct than drawing candlesticks by hand in React.
Provide deterministic mock/fallback data	Mock stocks, commodities, indices, generated candles, generated option chains, replay tapes, and synthetic greeks keep the UI useful when live providers are unavailable.	Fallback data can look real, so the UI and docs need clear distinction between live and simulated sources. The benefit is a demoable system without live credentials.

5. Deep Dive - Hardest Problems

Market-data latency and rate limits

The backend has to serve a UI with quote boards, chart refreshes, F&O views, portfolio LTP sync, diagnostics, and streaming without hammering providers. It handles this with async clients, connection pooling, short TTL caches, cache pruning, in-flight task dedupe, semaphores, cooldown state, stale-cache returns, and batch endpoints like `/api/workstation/snapshot` and `/api/ltp`.

Provider normalization and fallback correctness

Upstox and Groww responses differ in field names, status wrappers, auth behavior, instrument identifiers, and error modes. The backend normalizes quote fields like LTP, change, change percent, OHLC, volume, bid/ask, circuits, 52-week values, open interest, depth, timestamp, source, exchange, segment, and instrument key. The code prefers live data but can fall back provider-by-provider.

Order lifecycle correctness

A paper engine still needs real trading semantics. The app validates price and quantity, rejects invalid F&O lot multiples, reserves margin, locks pending sell quantity, respects market hours, queues pending stop/limit orders, executes partial fills, updates average price, reduces sells against existing positions, records charges, realized P&L, and changes order statuses.

Derivatives symbol resolution

F&O contracts are not friendly strings. The backend resolves simplified tickers using Upstox derivative indexes and `instruments.csv`, including underlying, expiry, strike, option type, future/option suffixes, and nearest non-expired contracts. The portfolio pages then resolve F&O positions before pulling LTP, with option-chain fallback if direct quote resolution fails.

Chart lifecycle and live patching

`StockChart` dynamically imports lightweight-charts, creates candlestick, histogram, and line-indicator series, observes container resize, caches candles, aggregates intervals including tick bars, formats all timestamps in IST, suppresses known disposal-race errors, and patches the latest candle from live LTP without recreating the chart.

Keeping a large React UI consistent

Prices can arrive from SSE, HTTP refetches, mock constants, portfolio polling, and user actions. The code uses a single stream provider, React Query for HTTP, localStorage-backed external store for portfolio data, browser events for command workflows, and request dedupe to prevent each card or page from independently flooding the backend.

Market-hours and holiday correctness

Both frontend and backend have IST market-session logic and holiday sets, including 2025/2026 data plus environment overrides. This controls status badges, polling/refetch intervals, SSE sleep durations, and order pending/executing semantics.

Security boundaries without full auth

The app has provider auth for Upstox/Groww but no end-user login. The code keeps provider credentials and runtime tokens on the backend, exposes missing-field diagnostics, and uses CORS restricted to localhost/dev origins. That is appropriate for local paper trading, but a production multi-user app would need real user auth, backend order persistence, and encrypted token storage.

ML/AI finding

No ML/AI components were found in this repository. There are no model files, training scripts, inference endpoints, embeddings, evaluation pipelines, or AI service clients. The closest features are deterministic **market replay**, moving-average replay/backtesting, generated mock candles, synthetic option-chain fallback calculations, and technical-indicator summaries; these are algorithmic simulations, not learned models.

6. Interview Questions and Answers

Question: Walk me through the architecture of EquityFlow end to end.

Answer: The app has a **Next.js/React** frontend and a **FastAPI** backend. The frontend renders the trading workstation, pages, command palette, charting, order pad, portfolio, strategy, diagnostics, and local paper account. The backend is a market-data gateway: it handles Upstox OAuth tokens, Groww authentication, quote/candle/depth/option-chain routes, provider fallback, caching, request coalescing, and SSE streams. User actions either mutate the local paper engine in **localStorage** or call backend HTTP/SSE endpoints. Streaming prices arrive through one shared **EventSource** provider, then components subscribe by ticker. The result is a local terminal-like desk where provider data and local simulated orders meet in the browser.

Concept tested: System decomposition, data flow, and separation of concerns.

Question: Why is FastAPI used instead of calling Upstox and Groww directly from the frontend?

Answer: The backend protects provider credentials and OAuth tokens, normalizes provider-specific responses, handles provider cooldowns, performs request coalescing, and streams a consolidated workstation feed. Calling providers from the browser would expose secrets, duplicate normalization logic, and make rate-limit handling harder because every card/page could generate independent requests. FastAPI also fits this code because the provider workload is I/O-heavy, so **asyncio** and **httpx.AsyncClient** are a natural match.

Concept tested: Backend-for-frontend design, security, and async I/O.

Question: Why did the project use Server-Sent Events instead of WebSockets?

Answer: The live-data flow is mostly one-way: the server pushes quote/status frames to the browser. **SSE** through **EventSource** is simpler than WebSockets, works over normal HTTP, has automatic reconnect behavior, and maps cleanly to FastAPI **StreamingResponse**. WebSockets would be useful if the server needed bidirectional order-entry commands, collaborative sessions, or low-level subscription negotiation. For this project, the client can express demand through HTTP and the shared stream can push prices back.

Concept tested: Streaming protocol choice and trade-off reasoning.

Question: How does the paper-trading engine avoid unrealistic order behavior?

Answer: The engine validates quantity, price, funds, held quantity, pending sell locks, F&O lot multiples, stop triggers, limit triggers, market hours, and circuit-like buffers. It supports **MARKET**, **LIMIT**, **SL**, and **SL-M** order varieties. Pending orders reserve margin; cancelled or modified orders release/adjust that margin; partial fills update remaining quantity; buys update weighted average positions; sells reduce holdings and calculate realized P&L. This makes it a domain model, not just a UI counter.

Concept tested: Domain modeling, correctness, and state transitions.

Question: How are provider rate limits handled?

Answer: The backend uses TTL caches per endpoint type, in-flight task maps to dedupe identical requests, semaphores to cap concurrent provider calls, Retry-After parsing, cooldown windows after 429s, stale-cache fallback, and diagnostics exposing cooldown state. The frontend also has path-aware request caching, in-flight dedupe, stale-on-429 returns, and cache pruning. Together these prevent a dense UI from generating a provider request per card per refresh.

Concept tested: Performance, resilience, cache design, and backpressure.

Question: How does F&O contract resolution work?

Answer: The backend builds indexes from Upstox instrument data and the local **instruments.csv** master. It supports simplified keys for options/futures by extracting underlying, expiry, strike, CE/PE/FUT type, and lot size, then chooses nearest non-expired contracts. The frontend can pass a simplified ticker like an option-like symbol; the backend resolves it to a provider trading symbol or instrument key before quote lookup. Portfolio components also poll F&O positions separately because the general SSE stream focuses on equities, commodities, and indices.

Concept tested: Data modeling for derivatives and symbol normalization.

Question: What is the persistence model, and what are its limitations?

Answer: Persistence is local-first. The paper account, positions, orders, transactions, watchlists, alerts, chart layouts, layout mode, and theme live in browser **localStorage**. Provider runtime tokens live in a backend token JSON file, and instrument metadata lives in CSV/provider indexes. This is ideal for a local paper-trading workstation because it is fast and simple. The limitation is no cross-device sync, no multi-user isolation, no server audit trail, and no transactional database guarantees. A production version should add backend account storage, likely PostgreSQL, and encrypted token storage.

Concept tested: Storage design, trade-offs, and production evolution.

Question: How is chart data loaded and updated?

Answer: The chart component fetches candles from `/api/candles/{ticker}`, caches them by ticker/timeframe/interval, transforms raw candles into time or tick buckets, and feeds candlestick/volume/indicator series into **lightweight-charts**. It formats time in IST, observes resize events, handles dark/light theme changes, and cleans up chart instances safely. When a live LTP is available, it patches the latest candle high/low/close instead of waiting for the next candle fetch.

Concept tested: Imperative UI integration, lifecycle cleanup, and real-time updates.

Question: What validation and error-handling patterns are visible?

Answer: Frontend order validation happens before mutation and includes invalid price/quantity, trigger price, insufficient funds, and lot-size checks. The backend uses **Pydantic** models, FastAPI **HTTPException**, provider response status checks, token refresh logic, exception capture, cooldowns, and fallbacks. The frontend request cache returns null or stale data when safe, while diagnostics and provider-status UI make degraded states visible rather than silently hiding failures.

Concept tested: Validation layers, error boundaries, and observability.

Question: How would you scale this project if real users depended on it?

Answer: First, move paper-account state from **localStorage** into a backend database with authenticated users and transaction-safe order tables. Second, centralize market-data fanout with a shared cache or stream broker so every user does not trigger provider calls. Third, use a scheduled instrument-master ingestion pipeline, durable token storage, and provider circuit breakers. Fourth, add end-to-end tests for order flows and streaming behavior. The current design already separates UI, provider proxy, caching, and engine logic, so those components can evolve independently.

Concept tested: Scalability path and system redesign under production constraints.

Question: What is the strongest technical decision in the codebase?

Answer: The strongest decision is separating live market-data concerns from simulated portfolio correctness. The backend handles provider instability, auth, normalization, and streaming, while the frontend engine handles deterministic paper-trading rules locally. That lets the UI remain responsive even if providers degrade, and it keeps the domain model testable through Vitest. It also makes interview discussion concrete because you can point to independent modules for provider resiliency, order correctness, charting, risk, and UI density.

Concept tested: Architecture judgment and modularity.

Question: What would you change first if you had more time?

Answer: I would add a backend persistence layer for user portfolios and orders, then unify the duplicate F&O position helper logic that appears in portfolio components. I would also clean up stale dependency/config drift, such as declared-but-unused packages and docs that mention older Next.js versions. After that, I would add browser end-to-end tests around the critical flows: provider connection, quote streaming, order placement, pending order trigger, cancel/modify, and portfolio P&L update.

Concept tested: Self-review, maintainability, and prioritization.

7. Follow-up Questions

Question: How do you know the code is not just a static mock dashboard?

Answer: Because the backend contains real provider integration, OAuth/token handling, HTTP clients, rate-limit/cooldown state, quote/candle/option-chain/depth routes, and SSE streaming. The frontend also has a real local order engine, pending-order processor, risk calculations, chart fetching, and tests. Mock data exists, but it is fallback/demo support, not the only data path.

Concept tested: Evidence-based project explanation.

Question: Where is the database schema?

Answer: There is no SQL or ORM schema. The data schema is split across TypeScript interfaces in **src/lib/types.ts**, Pydantic models in **backend/main.py**, browser localStorage records, provider token JSON, and **instruments.csv**. In an interview, call this a local-first paper-trading persistence model and acknowledge that production would need a backend database.

Concept tested: Honesty about implementation boundaries.

Question: What happens when the market is closed?

Answer: Market-hours utilities compute IST session state and holidays. The frontend adjusts status badges and refetch intervals, and the backend changes stream sleep intervals. Pending local orders can remain queued when market conditions do not allow execution. Mock/fallback data can still render the UI so the workstation stays usable for demos and practice.

Concept tested: Time-dependent correctness.

Question: How does the app prevent duplicate request storms?

Answer: There are two layers. In the browser, **request-cache.ts** dedupes in-flight GETs and applies TTLs by path. In the backend, provider GETs use cache maps and in-flight task maps keyed by path/params, plus semaphores. This matters because one route can render many stock cards, charts, positions, and diagnostics at once.

Concept tested: Load control and cache layering.

Question: Is this safe for real-money trading?

Answer: The browser-local engine is explicitly a paper-trading simulator. Backend Groww order proxy routes exist, but the main UX and OrderPad described here simulate trades locally and even labels orders as simulated paper trades. For real money, the project would need authenticated users, audited backend order state, stronger token custody, explicit broker-side confirmation flows, and compliance controls.

Concept tested: Security and product boundary awareness.

Question: What tests best prove the hard parts work?

Answer: The important tests are the Vitest suites for **engine.ts**, **risk-engine.ts**, **command-parser.ts**, **alerts.ts**, **replay.ts**, and **request-cache.ts**, plus the Python Upstox auth expiry unittest. They target domain behavior like order rejection, averaging, sell reductions, charges, realized P&L, lot-size validation, SL-M triggers, partial fills, margin/risk, parser commands, alert evaluation, and stale-on-429 request handling.

Concept tested: Testing strategy and risk-based coverage.

8. Quick Reference Card

Elevator pitch

EquityFlow is a local Indian-market paper-trading workstation that combines a dense **Next.js** terminal UI with a **FastAPI** market-data proxy. It streams and normalizes quotes from Upstox/Groww, displays live charts and option-chain style data, and lets a user simulate realistic trades with market, limit, stop-loss, partial fills, margin, charges, portfolio analytics, risk scores, watchlists, alerts, replay, and strategy payoff tools. The strongest part is that it treats trading as a real domain model rather than a static dashboard.

Five most impressive technical facts

- A single **EventSource** provider fans out live prices to many UI components without every component polling independently.
- The backend has real provider resilience: async HTTP connection pools, in-flight request coalescing, TTL caches, rate-limit cooldowns, stale-cache fallback, and diagnostics.
- The paper engine models complex order behavior: **MARKET**, **LIMIT**, **SL**, **SL-M**, pending orders, partial fills, margin reservation, sell locks, charges, and realized/unrealized P&L.
- The F&O path resolves simplified derivative tickers against provider/index data and a **205,894-row** instrument CSV rather than relying only on hardcoded symbols.
- The chart layer uses **lightweight-charts** with candle caching, interval aggregation, indicator series, IST formatting, live-LTP candle patching, and lifecycle cleanup.

Three things that stand out from typical student projects

- It includes operational visibility through provider status, diagnostics, cache/cooldown reporting, and degraded-state UI instead of assuming APIs always work.
- It has meaningful domain logic and tests for trading correctness, not just CRUD screens or static visualizations.
- The UI is designed as a dense workstation with command parsing, chart matrix presets, replay, strategy builder, risk panels, and paper account controls, which makes it feel like a tool rather than a demo page.

One-minute interview delivery

Start with the problem: practicing Indian-market trading workflows safely. Then describe the split architecture: Next.js terminal UI plus FastAPI provider proxy. Name the hardest pieces: streaming quotes without request storms, normalizing Upstox/Groww data, resolving F&O contracts, and making paper orders behave realistically. Close with what you would productionize: authenticated backend persistence, shared market-data fanout, durable instrument ingestion, and stronger end-to-end tests.